

프로그래머 포트폴리오

김석희

Email: klop2211@gmail.com

이력서

❖ 소개



이름: 김석희

연락처: 010-5875-4233

이메일: klop2211@gmail.com

생년월일: 2000/11/22

❖ 학력

2019.03 – 2026.02

한국공학대학교 게임공학과 수료

2016.03 – 2019.02

오산 운암고등학교 졸업

❖ 기술

C++

DirectX 12

HLSL

❖ 수상

2025.10.22

교내『2025년 AI·SW캡스톤디자인 경진대회』우수상

❖ 깃허브

<https://github.com/klop2211>

목차

- 프로젝트 개요
- CPU, GPU 동기화 문제 개선
 - 공유 자원 파악 및 FrameResource 설계
 - 원형 버퍼를 사용한 CPU, GPU 병렬 처리
- GPU 상태 변경 최소화를 위한 렌더링 최적화
 - PSO, Material, Mesh 순의 배치처리 구조
 - FPS 저하 현상 분석
 - 하드웨어 인스턴싱을 통한 드로우 콜 개선
 - 절두체 컬링 재적용 (Visible Indices)
- 컴포넌트 패턴 기반 게임오브젝트 프레임워크
 - 상속 기반 설계의 문제와 합성 패턴 적용
 - 순환 참조 방지를 위한 메모리 관리 설계

프로젝트 개요

- 제목: Sandy Heroes (모래의 영웅들)
- 장르: 캐주얼 협동 FPS
- 개발 기간: 2025.01 ~ 2025.08 (8개월)
- 개발 인원: 3인(클라이언트 2, 서버 1)
- 게임 흐름: 스테이지 진행 -> 총기 및 스크롤 획득 -> 최종 스테이지의 보스 처치 시 클리어
- 유튜브: <https://youtu.be/CVMWE64RcTc?si=GVuzDej7Bwjohhel>

인 게임 스샷

구현 내용

- DirectX 12기반 게임 프레임워크 구현
- 컴포넌트 패턴 형식의 게임 오브젝트 구현
- Skinned Mesh 애니메이션 구현
- 인게임 컷신 구현
- 몬스터, 플레이어 HP바 등 일부 UI 구현
- D3D11On12를 활용한 텍스트 렌더링 구현

Sandy Heroes

- CPU, GPU 동기화 문제 파악 및 해결책 설계

```
hResult = m_pd3dCommandList->Close();

ID3D12CommandList *ppd3dCommandLists[] = { m_pd3dCommandList };
m_pd3dCommandQueue->ExecuteCommandLists(1, ppd3dCommandLists);

WaitForGpuComplete();
```

▲ DirectX 12를 막 배울 당시 GameFramework의 동기화 구조

- 매 프레임마다 CPU, GPU 모두 서로의 작업이 모두 끝날 때까지 기다리는 비효율적인 구조를 가짐
- 왜냐하면, CPU, GPU가 **하나의 공유 자원**을 가지고 렌더링 작업을 수행했기 때문
- 따라서, 이를 개선하기 위해서는 **복수의 공유 자원**이 필요하다고 생각하였고 이를 위해 우측의 설계 과정을 거쳐 개발

1. CPU, GPU의 공유 자원 파악

공유 자원은 매 프레임마다 CommandList를 기록하기 위한 **CommandAllocator**와 게임 오브젝트 위치나 Skinned Mesh 애니메이션 등을 위한 **업로드 버퍼**들로 구성된다.

2. Fence를 통한 동기화 관리

복수의 공유 자원을 사용한다고 해도 여전히 CPU, GPU가 특정한 공유 자원 하나에 동시에 접근하면 안되기 때문에 여전히 **Fence를 통한 관리**가 필요하다.

```
ComPtr<ID3D12CommandAllocator> d3d_allocator;

std::unique_ptr<UploadBuffer<CBPass>> cb_pass;
std::unique_ptr<UploadBuffer<CBShadow>> cb_shadow;
std::unique_ptr<UploadBuffer<CBObject>> cb_object;
std::unique_ptr<UploadBuffer<CBBoneTransform>> cb_bone_transform;
std::unique_ptr<UploadBuffer<CBMaterial>> cb_material;
std::unique_ptr<UploadBuffer<CBUi>> cb_ui;

UINT64 fence = 0;
```

▲ 공유 자원을 담고 있는 FrameResource 구조체

Sandy Heroes

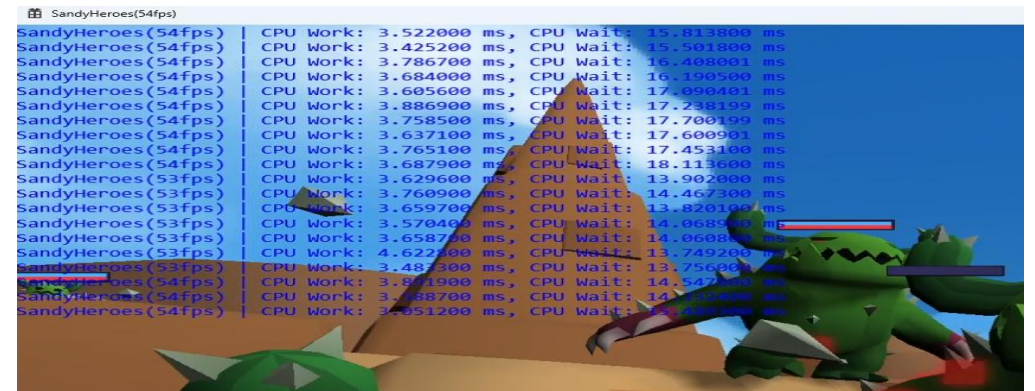
- CPU, GPU 동기화 문제 개선 및 성능 측정

- 공유 자원을 관리하기 위한 FrameResourceManager 클래스를 구현
- 공유자원은 이 클래스 내부에서 kFrameCount(상수값*) 크기의 원형 버퍼로 관리
*실제 게임에서 3으로 구현
- 이 클래스에서 공유 자원 버퍼의 초기화 및 CPU가 버퍼를 전부 사용했을 때 CPU, GPU 동기화를 진행

```
void FrameResourceManager::CirculateFrameResource(ID3D12Fence* fence)
{
    // 다음 프레임 리소스 인덱스로 이동 (Ring Buffer)
    curr_frame_resource_index_ = (curr_frame_resource_index_ + 1) % kFrameCount;
    curr_frame_resource_ = frame_resources[curr_frame_resource_index_].get();

    // GPU가 해당 프레임 리소스 사용을 완료했는지 체크
    // fence 값이 0이 아니고(초기상태 제외), GPU의 완료 값(GetCompletedValue)이 목표 값보다 작다면 대기
    if (curr_frame_resource_>fence != 0 &&
        fence->GetCompletedValue() < curr_frame_resource_>fence)
    {
        HANDLE event_handle = CreateEventEx(nullptr, nullptr, 0, EVENT_ALL_ACCESS);
        fence->SetEventOnCompletion(curr_frame_resource_>fence, event_handle);
        WaitForSingleObject(event_handle, INFINITE);
        CloseHandle(event_handle);
    }
}
```

▲ 원형 버퍼를 사용한 CPU, GPU 동기화



▲ CPU 작업시간 및 대기시간 측정(kFrameCount = 1)

CPU 평균 작업시간을 3.66ms, GPU 평균 작업시간을 15.32ms로 가정

1. 공유자원이 1개인 경우
 $1000\text{ms} / (3.66\text{ms} + 15.32\text{ms}) = \text{약 } 52.7\text{fps}$
2. 공유자원이 3개인 경우
 $1000\text{ms} / 15.32\text{ms} = \text{약 } 62.3\text{fps}$

약 9.6fps만큼의 차이가 났으며 약 18.2%의 성능향상을 측정

Sandy Heroes

- GPU 상태 변경 최소화를 위한 배치처리

- 렌더링 구조 개발 당시 **GPU 상태 변경으로 인한 오버헤드**를 고려하여 PSO(Shader), DescriptorTable(Texture) 순서로 **상태 변경을 최소화하기 위한 구조**를 프레임워크에 구현
- Shader, Material 각 클래스가 본인을 사용하는 materials_, mesh_components_를 관리하기 때문에 별도의 오브젝트 정렬을 요구하지 않는 구조로 개발
- Scene을 렌더할 때 전체 PSO의 개수만큼 PSO가 변경되며 동일한 Material 사용하는 Mesh들에 대해서 한번의 DescriptorTable 변경이 일어남

1

```
void Scene::Render(ID3D12GraphicsCommandList* command_list)
{
    FrameResourceManager* frame_resource_manager = game_framework->frame_resource_manager();
    auto curr_frame_resource = frame_resource_manager->curr_frame_resource();

    for (const auto& [type, shader] : shaders_)
    {
        shader->Render(command_list, curr_frame_resource, game_framework->descriptor_manager(), main_camera_);
    }
}
```

▲ Scene에서 PSO를 가진 Shader 호출

2

```
void Shader::Render(ID3D12GraphicsCommandList* command_list,
    FrameResource* curr_frame_resource, DescriptorManager* descriptor_manager, CameraComponent* camera, bool bShadow)
{
    if(!bShadow)
        command_list->SetPipelineState(d3d_pipeline_state_.Get());

    for (const auto& const material : materials_)
    {
        material->Render(command_list, curr_frame_resource, descriptor_manager, camera, bShadow);
    }
}
```

▲ PSO를 Set하고 해당 PSO를 사용하는 Material들을 호출

3

```
void Material::Render(ID3D12GraphicsCommandList* command_list,
    FrameResource* curr_frame_resource, DescriptorManager* descriptor_manager,
    CameraComponent* camera, bool bShadow)
{
    if(!bShadow)
        //텍스처 및 머터리얼 상수버퍼 설정
        UpdateShaderVariables(command_list, curr_frame_resource, descriptor_manager);

    for(const auto& mesh_component : mesh_components_)
    {
        mesh_component->Render(this, command_list, curr_frame_resource);
    }
}
```

▲ 동일한 Material을 사용하는 Mesh들을 렌더

Sandy Heroes

- FPS 저하 현상 파악 및 PIX를 통한 분석

- 게임을 테스트할 때 간헐적으로 프레임이 심하게 저하되는 현상을 확인
- 여러 차례 플레이 결과 2가지 구체적인 상황을 특정
 - 레이저나 총기 피격에 의한 **파티클이 다수 생성될 때**
 - 맵이 크고 **오브젝트가 많은 곳**을 바라 볼 때
- 1번 상황의 경우 몬스터나 파티클에 의해 화면이 계속 바뀌기 때문에 fps가 급격히 변화
- 따라서, fps 측정이 용이한 2번 상황을 분석
- 인게임 fps의 경우 49fps 정도 나왔으며 원인 분석을 위해 PIX를 사용하여 분석한 결과 약 1800회의 드로우 콜 횟수를 확인
- 약 1800회의 드로우 콜 중에서 대부분의 드로우 콜이 지형지물을 구성하는 단순 Mesh인 것을 파악



▲ 오브젝트가 많은 맵에서 측정된 인게임 fps

Global ID	Name
1,883	DrawIndexedInstanced(120,1,0,0,0) {this->ID3D12GraphicsCommandList obj#5}
1,884	DrawIndexedInstanced(120,1,0,0,0) {this->ID3D12GraphicsCommandList obj#5}
1,885	DrawInstanced(91,1,0,0) {this->ID3D12GraphicsCommandList obj#5}
1,886	DrawInstanced(93,1,0,0) {this->ID3D12GraphicsCommandList obj#5}
1,887	DrawInstanced(96,1,0,0) {this->ID3D12GraphicsCommandList obj#5}
1,888	DrawInstanced(95,1,0,0) {this->ID3D12GraphicsCommandList obj#5}
1,889	DrawInstanced(92,1,0,0) {this->ID3D12GraphicsCommandList obj#5}
1,890	UpdateSubresource on GPU timeline
1,891	PreDraw
1,892	DrawIndexedInstanced(6,366,0,0,0) {this->ID3D12GraphicsCommandList obj#375}
1,893	ResourceBarrier(1,...) {this->ID3D12GraphicsCommandList obj#375}
1,894	Signal(obj#390,3011) {this->ID3D12CommandQueue obj#1,return->S_OK}
1,895	Signal(obj#393,3010) {this->ID3D12CommandQueue obj#1,return->S_OK}
1,896	Present(obj#9,0,<unknown>) {this->ID3D12SharingContract obj#1}

▲ 약 1800회 이상 드로우 콜 호출

Sandy Heroes

- 드로우 콜 개선을 위한 하드웨어 인스턴싱 설계

- 플레이어 캐릭터나 몬스터 같이 Skinned Mesh가 아닌 지형 지물 같은 일반적인 단순한 Mesh의 경우 동일한 메쉬를 여러 번 그리는 경우가 많음
- 이러한 Mesh에 하드웨어 인스턴싱을 적용하면 **동일한 Mesh를 1회의 드로우 콜로 렌더**할 수 있기 때문에 기존 Mesh 렌더링 구조에 적용

```
void Mesh::Render(ID3D12GraphicsCommandList* command_list, int material_index)
{
    command_list->IASetVertexBuffers(0, ...);
    command_list->IASetIndexBuffer(&index_buffer_views_[material_index]);
    // 인스턴스 개수 = 1 (오브젝트마다 이 함수가 호출됨)
    command_list->DrawIndexedInstanced(index_count, 1, 0, 0, 0);
}
```

▲ Mesh 1개 렌더할 때마다 드로우 콜이 1회 발생*

* 인스턴싱 횟수를 1로 하드코딩 하여 사용했던 이유는
렌더링 구조를 개발할 때 하드웨어 인스턴싱을 고려하지 않았기 때문

- 기존 렌더링 구조에 하드웨어 인스턴싱을 적용하기 위해 3가지 과정을 설계

1. 인스턴싱 데이터를 위한 StructuredBuffer

하드웨어 인스턴싱을 구현하기 위해선 Mesh들을 렌더하는데 필요한 월드행렬들을 **연속된 인덱스로 접근 가능한 버퍼**에 담아야 한다.

이를 위해 StructuredBuffer를 사용하는 것이 적합하다고 생각했다.

2. 오브젝트의 업로드 버퍼 업데이트 구조의 수정

StructuredBuffer의 데이터는 동일 메쉬를 사용하는 오브젝트가 모여 있어야 한다.

따라서, **동일 Mesh를 사용하는 오브젝트들을 파악할 수 있는 구조**의 필요성을 느꼈다.

3. Mesh::Render 함수에서 사용할 인스턴스 횟수 파악

Mesh::Render의 대부분의 코드는 하드웨어 인스턴싱과 무관하기 때문에

이 함수를 호출하는 부분에서 인스턴스 횟수를 파악만 하면

큰 수정 없이 그대로 사용할 수 있다고 생각했다.

Sandy Heroes

- 하드웨어 인스턴싱 구현 과정

1. 인스턴싱 데이터를 위한 StructuredBuffer

- 인스턴스 데이터를 담기 위한 StructuredBuffer 선언 및 MeshVS를 수정하여 인스턴스 인덱스로 StructuredBuffer에서 오브젝트의 월드행렬 참조
- StructuredBuffer를 위한 업로드 버퍼 추가 및 해당 버퍼를 관리하기 위한 offset* 추가

*업로드 버퍼의 GPUAddress를 offset 통해 관리

2. 오브젝트의 업로드 버퍼* 업데이트 구조의 수정

- 기존에는 Scene에서 수행하던 업로드 버퍼 업데이트를 Material에서 해당 Material과 연결된 mesh_components를 Mesh순으로 정렬하여 동일 Mesh끼리 분류한 뒤 순차적으로 업로드 버퍼를 업데이트하도록 구현

*오브젝트의 업로드 버퍼는 연결된 MeshComponent 클래스에서 업데이트

3. Mesh::Render 함수에서 사용할 인스턴스 횡수 파악

- Mesh와 해당 Mesh를 사용하는 MeshComponent 컨테이너를 MeshInstanceInfo 구조체로 구현

```
struct InstanceData
{
    matrix world_matrix;
    float object_time;
};

StructuredBuffer<InstanceData> g_instance_data : register(t0, space1);

VertexOut MeshVS(MeshVertexIn v_in, uint instance_id : SV_InstanceID)
{
    VertexOut v_out;

    matrix world = g_instance_data[instance_id].world_matrix;
```

HLSL 코드 수정

```
struct InstanceData
{
    XMFLLOAT4X4 world_matrix;
    float time;
};

// 게임에서 한 프레임에 사용하는 리소스에 대한 구조체
struct FrameResource
{
public:
    std::unique_ptr<UploadBuffer<InstanceData>> sb_instance_data;
    int current_instance_offset = 0;
```

FrameResource 구조체 수정

Sandy Heroes

- 하

cbuffer Obj

```
{  
    matrix  
    float g  
};
```

VertexOut M

```
{  
    float4  
    // ...  
}
```

void Mesh::I

```
{  
    command_  
    command_  
    // 인스턴  
    command_  
}
```

이전 페이지 요약: 하드웨어 인스턴싱을 구현 하는 과정 설명

현재 페이지에 들어갈 것: 하드웨어 인스턴싱을 통해 얻은 성능 향상에 대한 분석

1. 인게임 FPS 비교(사진 2개)

2. PIX를 통한 드로우 콜 횟수 비교(사진 2개)

3. 하드웨어 인스턴싱 적용 과정에서 문제점

- 동일 메쉬를 사용하는 MeshComponent를 모아서 렌더하는 과정에서 카메라 절두체 컬링이 무시되었음
왜: Material의 mesh_component_list_는 단순히 해당 Material을 사용하는 모든 MeshComponent를 가지고 있기 때문에 해당 컨테이너를 활용해서 업로드 버퍼와 인스턴스 횟수 측정을 하는 과정에서 절두체 컬링이 무시 되었다.

Sandy Heroes

- 하

```
void  
{  
  ...  
}
```

하드
사용
류어

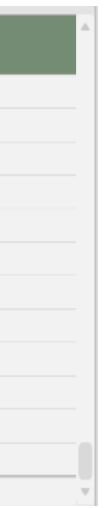
PIX

- 모
- 유

하드웨어 인스턴싱을 통해 DrawCall 오버헤드가 크게 줄어 CPU 부하가 감소하였고, GPU 파이프라인이 더 효율적으로 활용될 수

이전 페이지 요약: 하드웨어 인스턴싱을 구현 하는 과정 설명
현재 페이지에 들어갈 것: 하드웨어 인스턴싱을 통해 얻은 성능 향상에 대한 분석

1. 인게임 FPS 비교(사진 2개)
2. PIX를 통한 드로우 콜 횟수 비교(사진 2개)



Sandy Heroes

- 절

이전
과정
바뀌
되었

이전 페이지 요약: 하드웨어 인스턴싱을 통해 얻은 성능 향상에 대한 분석
현재 페이지에 들어갈 것: 하드웨어 인스턴싱을 적용하면서 생긴 문제점과 해결법 제시

1. 하드웨어 인스턴싱 적용 과정에서 문제점

- 동일 메쉬를 사용하는 MeshComponent를 모아서 렌더하는 과정에서 카메라 절두체 컬링이 무시되었음

왜: Material의 mesh_component_list_는 단순히 해당 Material을 사용하는 모든 MeshComponent를 가지고 있다. 그 컨테이너를 활용해서 업로드 버퍼를 업데이트하고 인스턴스 횟수 측정하였기 때문이다.

2. Visible indices를 추가하여 해결하였다.

Visible indices란?

카메라 절두체 컬링을 통과한 오브젝트의 g_instance_data의 인덱스를 g_main_visible_indices 배열에 추가 메인 렌더 시 SV_InstanceID가 g_main_visible_indices를 참조하여 g_instance_data에 접근하는 방법이다. 이를 통해 기존의 업로드 버퍼를 업데이트 하는 로직은 건들지 않은 채 절두체 컬링을 적용하였다. 또 한 더 나아가서 그림자 영역 판정에도 같은 방식을 적용하여 Shadow 렌더 패스에서 그림자 영역 안에 있는 오브젝트만 렌더하게 개선하였다.

Sandy Heroes

- 절

공유
구현
kFr

원형

이름
사용

```
void  
{  
  ~~~~~  
  ~~~~~  
  ~~~~~  
}
```

이전 페이지 요약: 하드웨어 인스턴싱을 적용하면서 생긴 문제점과 해결법 제시
현재 페이지에 들어갈 것: 절두체 컬링 재적용에 따른 성능향상 체크

1. 인게임 FPS 비교(사진 2개)
2. PIX를 통한 드로우 콜 횟수 비교(사진 2개)

State ▾

unters

.가
411

Sandy Heroes

- 절

공유
구현
kFr

원형

이름
사용

```
void  
{  
-  
-  
-  
}
```

이전 페이지 요약: 절두체 커플링 재적용에 따른 성능향상 체크
현재 페이지에 들어갈 것: 게임 오브젝트 구현 과정에서의 문제와
해당 문제를 컴포넌트 패턴 구현으로 해결한 이유
여러가지 텀 프로젝트를 진행하면서 게임 오브젝트 클래스들을 개발 할 때
클래스 내부의 기능들이 서로 커플링 되어 클래스 재사용에 어려움을 겪었다.
이를 해결하기 위해 오브젝트와 각 기능들을 컴포넌트로 분리하여
클래스로 구현하는 컴포넌트 패턴을 게임 오브젝트에 적용하였다.
(다이어그램 설명 2개, Object 헤더 코드 1개)

State ▾

unters

가
411

Sandy Heroes

- 절

공유
구현
kFr
원형
이름
사용

```
void  
{  
  ~~~~~  
  ~~~~~  
  ~~~~~  
}
```

이전 페이지 요약: 컴포넌트 패턴 구현 및 구현의 이유
현재 페이지에 들어갈 것: 오브젝트 컴포넌트 리팩토링

State ▾

unters

.가
411